

THE MATCOM MINIX

Facultad de Matemática y Computación
Universidad de La Habana

Introducción

El presente proyecto tiene como objetivo el aprendizaje práctico y progresivo de los principios fundamentales de los Sistemas Operativos. MINIX se utilizará como plataforma de trabajo para implementar, modificar y analizar componentes esenciales como procesos, planificación y sistemas de archivos.

Orientaciones generales

El proyecto integra progresivamente algunos de los principales componentes de un sistema operativo real, permitiendo modificar, extender y comprender internamente su funcionamiento. Se podrá realizar individual o en dúos. No obstante, ambos integrantes deben dominar la totalidad del contenido (tanto del curso como del proyecto propio), ya que durante la evaluación se realizarán preguntas sobre cualquiera de los temas abordados. Asimismo, se debe aclarar que la defensa será presencial, o virtual únicamente en casos excepcionales. En ambos casos, los dos estudiantes deberán encontrarse en la facultad o participar conjuntamente en una misma sesión virtual.

Se adjunta además un cronograma con hitos a alcanzar durante el desarrollo. Estos hitos tienen carácter evaluativo.

El código fuente del sistema operativo debe quedar en un repositorio de GitHub de sus cuentas personales, forkeado desde el repositorio indicado, al cual se le hará un Pull Request para la entrega final.

Además de la implementación práctica, se debe entregar también un informe técnico, elaborado de acuerdo con la planilla que se les envía, con el fin de documentar las decisiones de diseño, los cambios realizados, las pruebas efectuadas y los resultados obtenidos.

1. Instalación y configuración de VirtualBox y MINIX

Instalar y configurar Oracle VirtualBox y crear una máquina virtual que ejecute la última versión estable de MINIX, siguiendo la documentación oficial del proyecto.

2. Personalización del mensaje de bienvenida

Modificar el archivo `/etc/motd` en MINIX para que, al iniciar sesión, se muestre un mensaje de bienvenida que incluya una referencia a la Facultad de Matemática y Computación de la Universidad de La Habana.

3. Depuración de un bug en pthread

En esta versión de MINIX, la implementación principal de hilos reside en `libmthread`, mientras que las funciones `pthread_*` se exponen mediante una capa de compatibilidad. En ese contexto, se ha identificado un fallo en `pthread_mutex_trylock`, implementada en `minix/lib/libmthread/pthread_compat.c`.

El síntoma observable es que un programa mínimo que invoque `pthread_mutex_trylock` sobre un mutex y repita la llamada queda bloqueado antes de completar las operaciones posteriores de `unlock` y `destroy`. En términos prácticos, la llamada no retorna y el programa parece colgado.

Se debe reproducir el fallo, localizar la función donde se origina, comprender la relación entre `pthread_*` y `mthread_*`, identificar la causa exacta del problema, proponer una corrección mínima y verificar experimentalmente su validez.

Como criterio de validación, tras corregir el bug, la primera llamada a `pthread_mutex_trylock` debe devolver 0, la segunda debe devolver `EDEADLK`, y las operaciones `pthread_mutex_unlock` y `pthread_mutex_destroy` deben completarse correctamente. El programa de prueba debe finalizar reportando éxito.

Programa de prueba de referencia

```
#define _MTHREADIFY_PTHREADS

#include <minix/mthread.h>

#include <errno.h>
#include <stdio.h>
#include <string.h>

static const char *errname(int err)
{
    if (err == 0)
        return "OK";

    return strerror(err);
}

int main(void)
{
    pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
    int r1, r2, r3, r4;

    r1 = pthread_mutex_trylock(&mutex);
    printf("first trylock: %d (%s)\n", r1, errname(r1));

    r2 = pthread_mutex_trylock(&mutex);
    printf("second trylock: %d (%s)\n", r2, errname(r2));

    r3 = pthread_mutex_unlock(&mutex);
```

```

printf("unlock: %d (%s)\n", r3, errname(r3));

r4 = pthread_mutex_destroy(&mutex);
printf("destroy: %d (%s)\n", r4, errname(r4));

if (r1 != 0)
return 1;
if (r2 != EDEADLK && r2 != EBUSY)
return 2;
if (r3 != 0)
return 3;
if (r4 != 0)
return 4;

printf("PASS\n");
return 0;
}

```

Salida esperada tras la corrección

```

first trylock: 0 (OK)
second trylock: 35 (Resource deadlock avoided)
unlock: 0 (OK)
destroy: 0 (OK)
PASS

```

4. Implementación del comando tree

Implementar un nuevo comando de usuario llamado **tree** que imprima recursivamente la estructura de directorios a partir de una ruta dada, similar al comando **tree** en otros sistemas.

Requisitos

1. Funcionalidad del comando

El comando debe ejecutarse como:

```
tree <ruta>
```

Si no se especifica ruta, debe asumir el directorio actual (.).
Debe listar:

- Directorios y subdirectorios de forma recursiva.
- Archivos contenidos en cada directorio.

Debe mostrar el nivel de profundidad mediante indentación.

3. Restricciones

- No se permite usar herramientas externas para generar el árbol.
- Debe implementarse mediante llamadas al sistema: `opendir`, `readdir`, `closedir`, `stat`, etc.
- Evitar ciclos: si se encuentran enlaces simbólicos, no deben seguirse recursivamente.

5. Penalización por uso intensivo de CPU

Modificar el planificador de MINIX para implementar un mecanismo sencillo de ajuste de prioridad basado en el uso sostenido de CPU dentro de ventanas de tiempo.

La idea central consiste en distinguir entre procesos que consumen CPU de manera continua y procesos de comportamiento más interactivo o bloqueante. Para ello, el scheduler deberá observar cuántas veces un proceso agota completamente su quantum dentro de una ventana de planificación, penalizando temporalmente a los procesos *CPU-bound* y permitiendo la recuperación gradual de aquellos que dejan de mostrar ese patrón de ejecución.

La implementación debe realizarse sobre el servicio `sched` de MINIX, modificando su lógica interna de planificación y, de ser necesario, la estructura de datos asociada a los procesos planificables.

Requisitos

1. Análisis previo

Antes de modificar el código, el estudiante deberá:

- Identificar en qué archivo del scheduler se gestiona el agotamiento del quantum.
- Determinar dónde se almacenan los datos de planificación de cada proceso.
- Comprender el papel de funciones como `do_noquantum`, `balance_queues`, `do_start_scheduling` y `do_stop_scheduling`.
- Reconocer cómo se representan la prioridad actual, la prioridad máxima permitida y el quantum asignado a cada proceso.

2. Modificación del scheduler

La solución deberá incorporar una política basada en ventanas temporales de planificación. En particular:

- Mantener, para cada proceso, un contador de quantums completos consumidos dentro de la ventana actual de planificación.
- Incrementar dicho contador cada vez que el proceso agote completamente su quantum y el scheduler reciba la notificación correspondiente.
- Si un proceso consume al menos N quantums completos dentro de una misma ventana (por ejemplo, $N = 3$), reducir su prioridad en un nivel, sin exceder los límites definidos por el sistema.

- Asegurar que la penalización no lleve la prioridad por debajo de la peor prioridad permitida para procesos de usuario.
- Ejecutar periódicamente un balanceo global del scheduler mediante el mecanismo ya existente de temporización.
- Si durante una ventana un proceso no consume ningún quantum completo, considerar que no se comportó como proceso intensivo en CPU y restaurar gradualmente su prioridad, sin superar su prioridad máxima permitida.
- Reiniciar el contador de quantums al finalizar cada ventana de balance.

3. Validación experimental

La modificación debe validarse mediante pruebas controladas. Se espera que el estudiante diseñe y ejecute al menos dos tipos de escenarios:

- Un proceso claramente *CPU-bound*, por ejemplo un bucle infinito sin operaciones bloqueantes.
- Un proceso con comportamiento interactivo o bloqueante, por ejemplo uno que duerma periódicamente o realice operaciones que cedan CPU.

Durante la validación deberán observarse, al menos, los siguientes aspectos:

- Evolución de la prioridad de ambos procesos a lo largo del tiempo.
- Diferencia de tratamiento entre un proceso intensivo en CPU y uno que no agota quantums de manera sostenida.
- Coherencia entre el comportamiento observado y la política implementada.